Volcano for the Agentic AI Era:

Unified Scheduling Across Training, Inference, and Agents

Ricardo Aravena · Rainie Li

KubeCon + CloudNativeCon North America 2026

Salt Lake City · November 9–12

Speakers

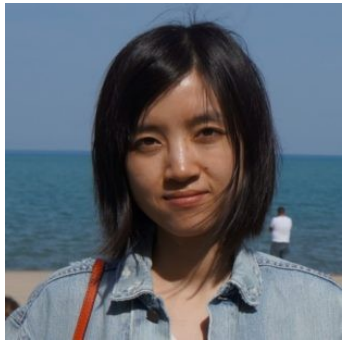


Ricardo Aravena

AI Infrastructure Lead · Nunchux AI

CNCF TOC Alternate (GB)
Cloud native and AI infrastructure

nunchux.ai · CNCF TOC



Rainie Li

Pinterest

Data processing infrastructure
Apache YuniKorn PMC member

Apache YuniKorn · rainieli

01

Unified scheduling

Different paths. Shared infrastructure.

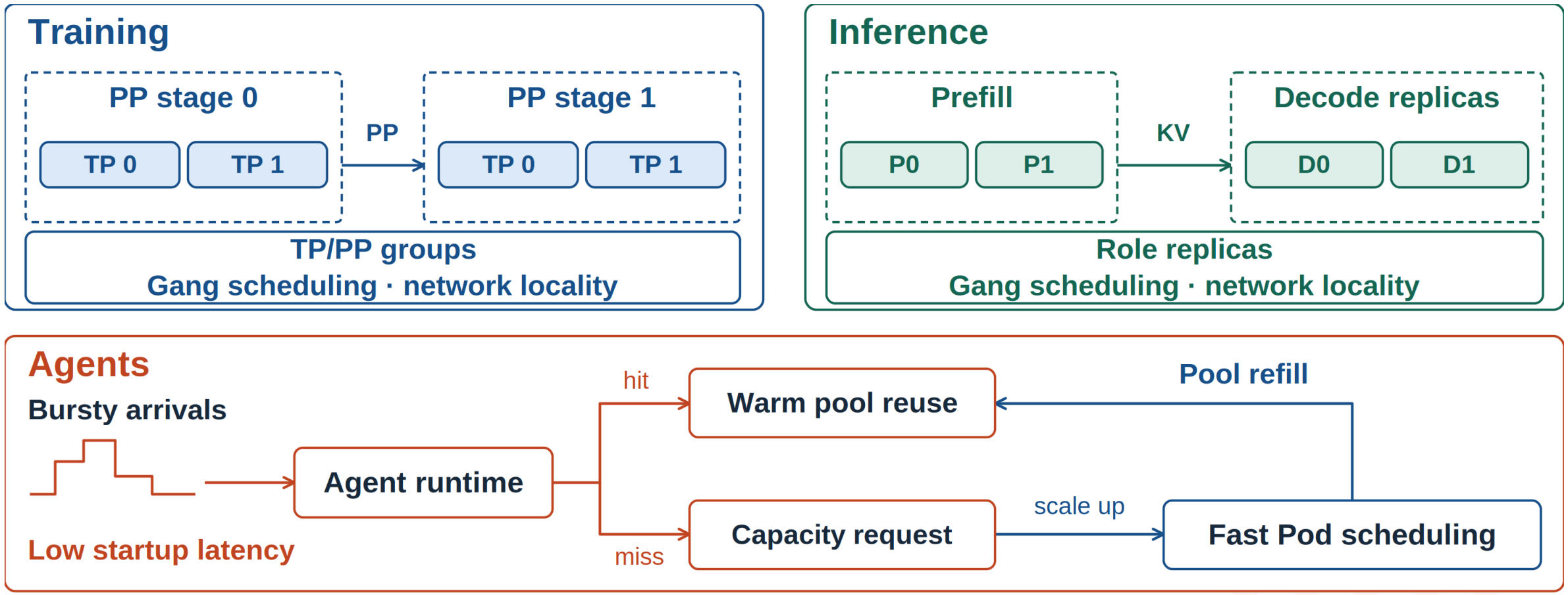
01 Unified scheduling

02 Multi-level gang & network topology

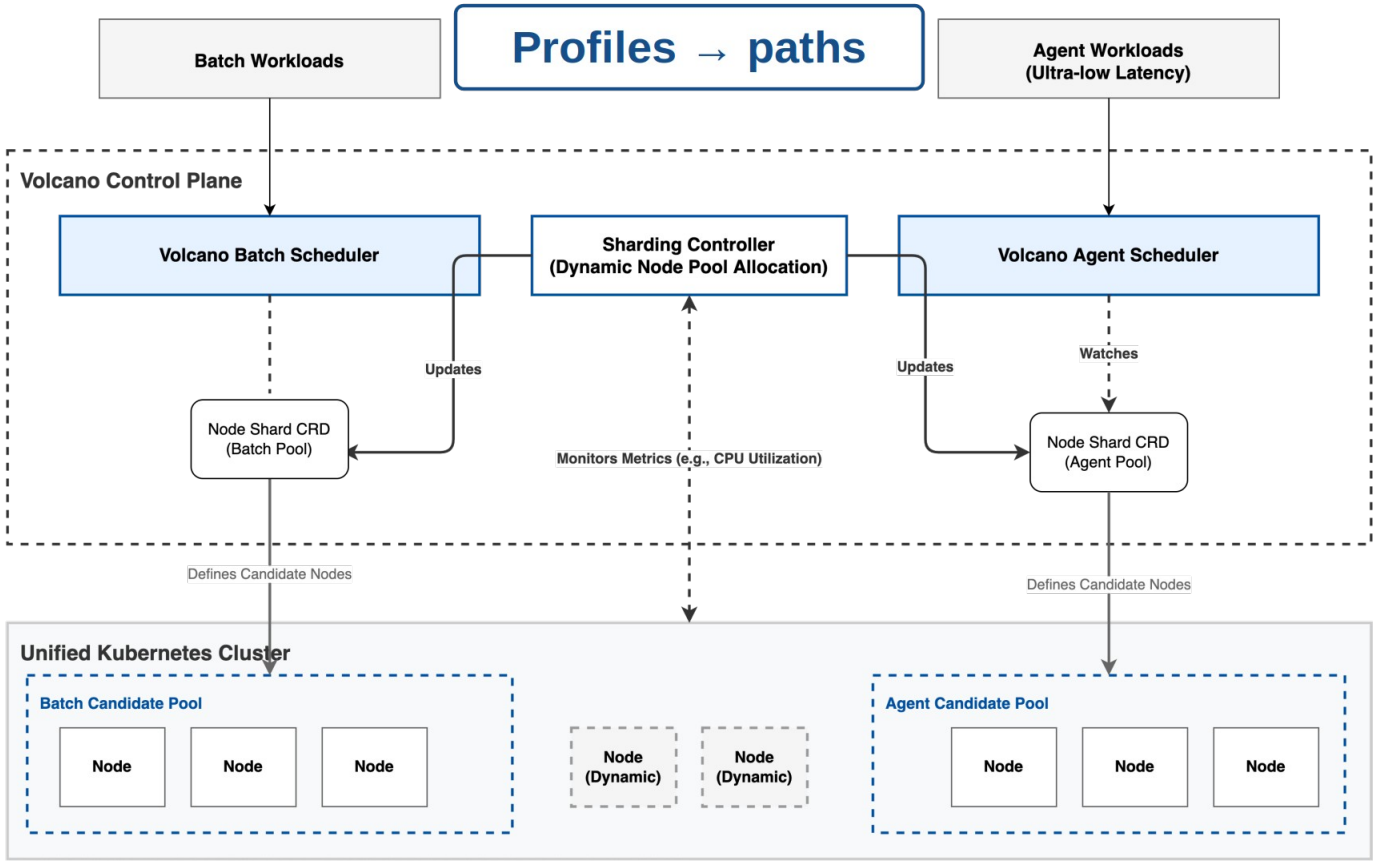
03 Ecosystem

04 Benchmark framework & progress

Different Workloads, Shared Infrastructure



A Coordinated Scheduling Framework



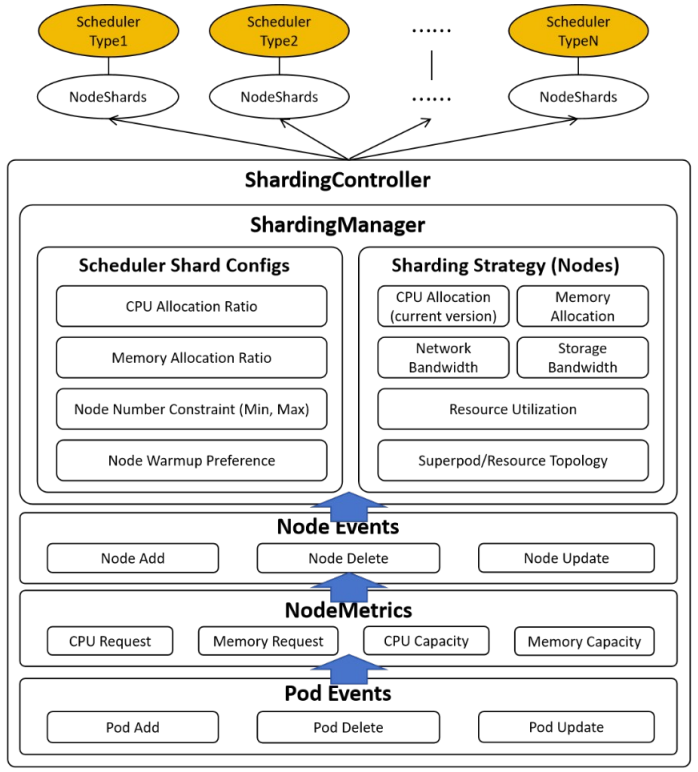
Sharding Controller

Live node state
Dynamic pool allocation

NodeShard

Candidate nodes
for each scheduling path

NodeShard: Strategy-Driven Node Allocation



Node metrics + strategy
Compute desired node allocation

NodeShard · spec / status

```
spec:
  nodesDesired:
  - node1
  - node2
status:
  nodesInUse:
  - node0
  - node1
  nodesToRemove:
  - node0
  nodesToAdd:
  - node2
```

Strategy result
Desired nodes

Scheduler state
Nodes in use

02

Multi-level gang & network topology

From available nodes to runnable workloads.

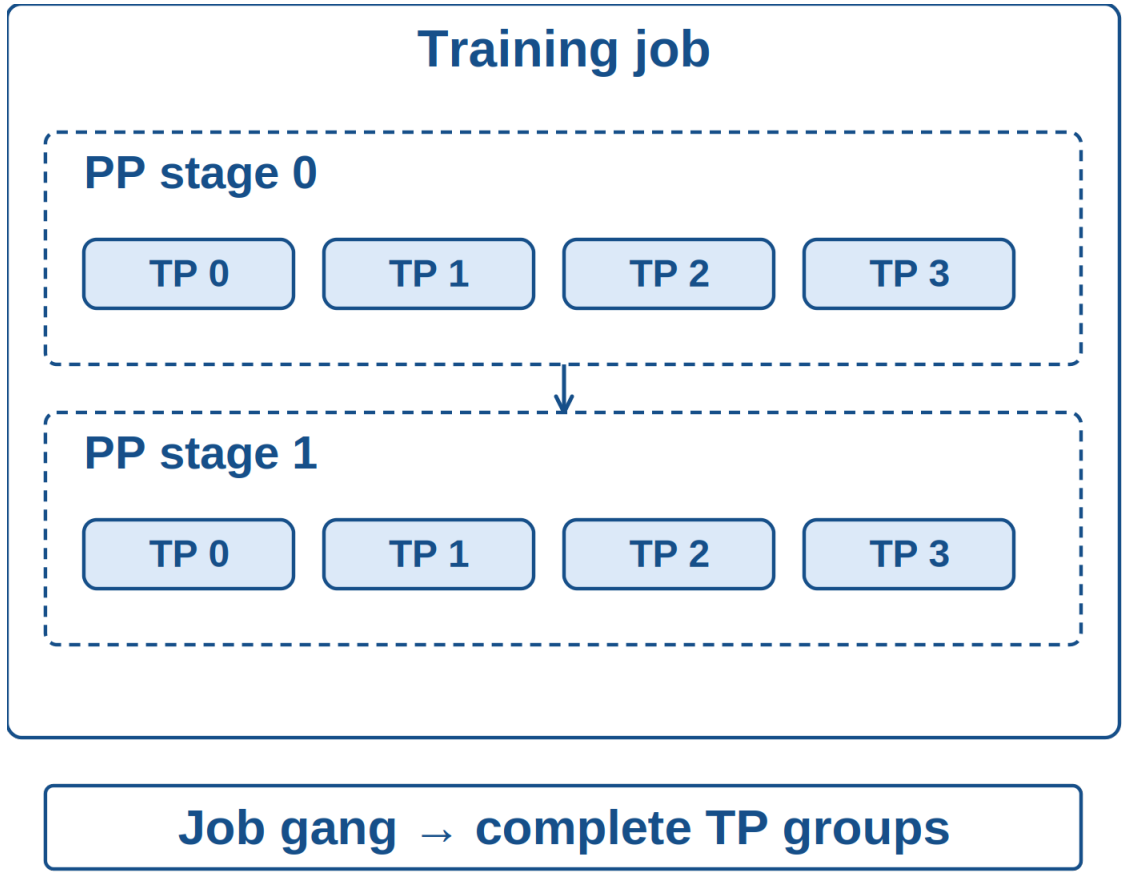
01 Unified scheduling

02 Multi-level gang & network topology

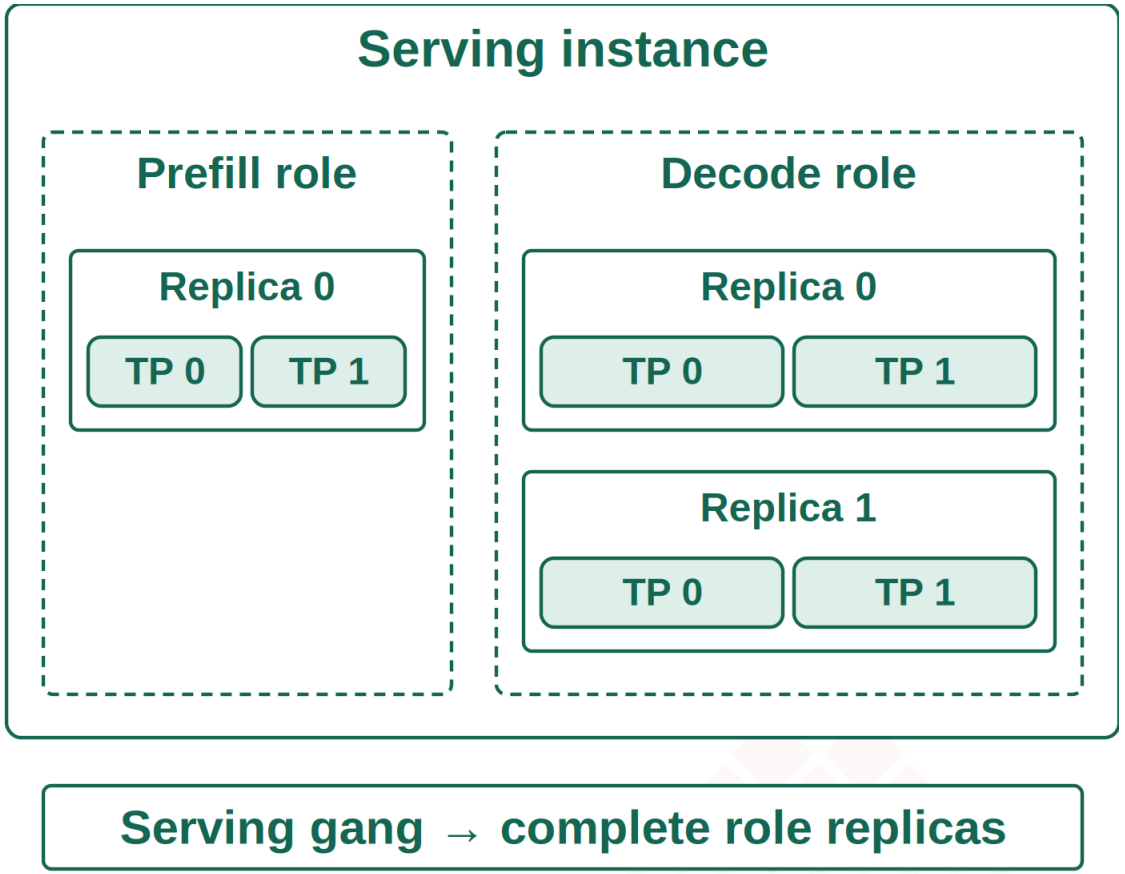
03 Ecosystem

04 Benchmark framework & progress

Whole Workloads, Complete Compute Groups



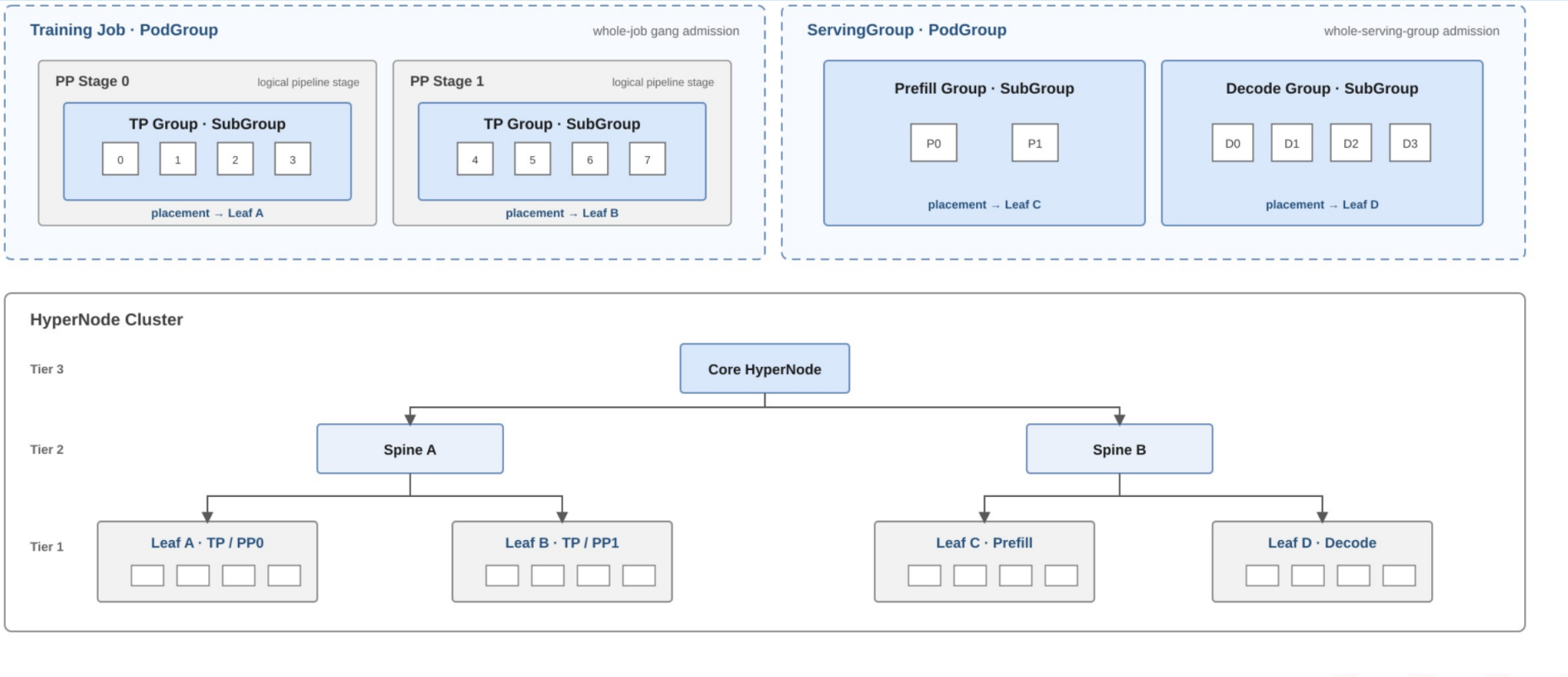
Job gang → complete TP groups



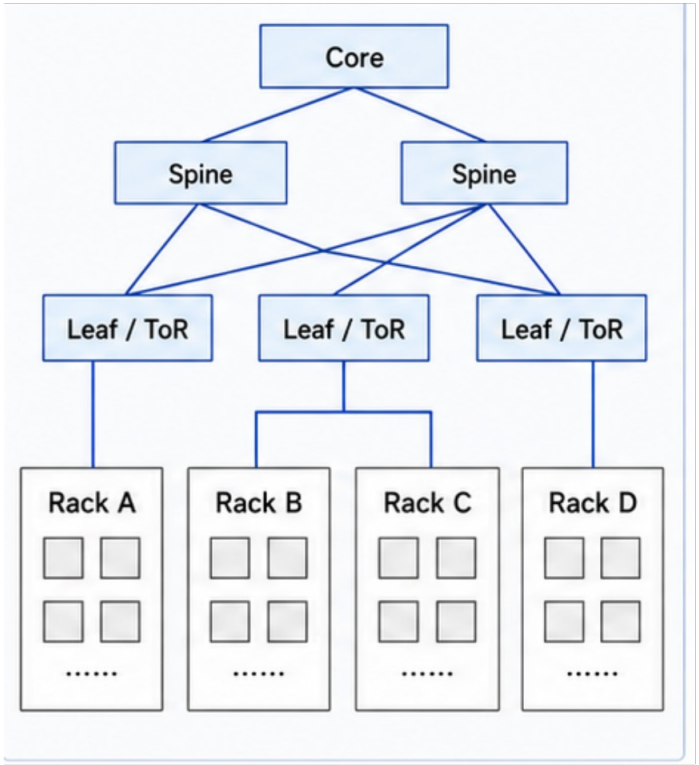
Serving gang → complete role replicas

PodGroup and SubGroup: Preserve Workload Structure

PodGroup: job / serving gang | SubGroup: TP / role replica gang



HyperNode: From Physical Fabric to Scheduling Domains



Parent HyperNode.spec

```
tier: 2
tierName: spine
members:
- type: HyperNode
  selector:
    exactMatch:
      name: rack-a
```

Groups child HyperNodes

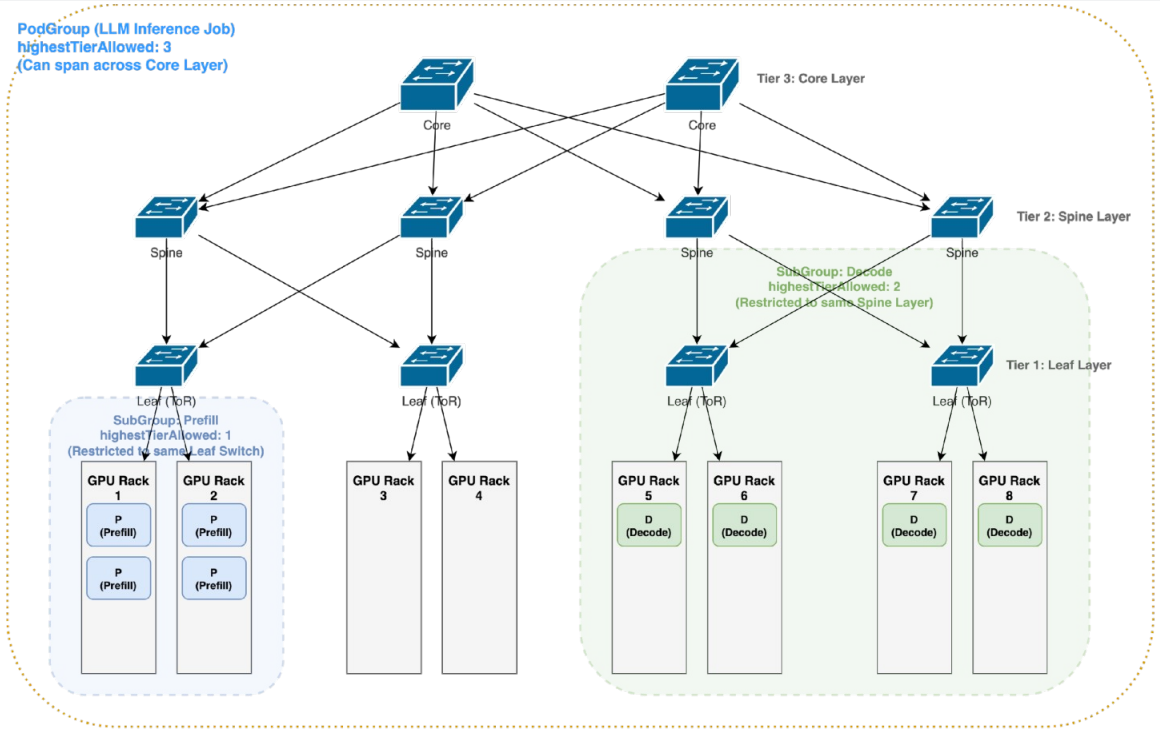
Leaf HyperNode.spec

```
tier: 1
tierName: leaf
members:
- type: Node
  selector:
    labelMatch:
      matchLabels:
        rack: rack-a
```

Groups Kubernetes nodes

Place Groups at Different Topology Levels

PodGroup · whole-instance placement at core



Prefill · leaf locality

Decode · spine locality

PodGroup.spec · key fields

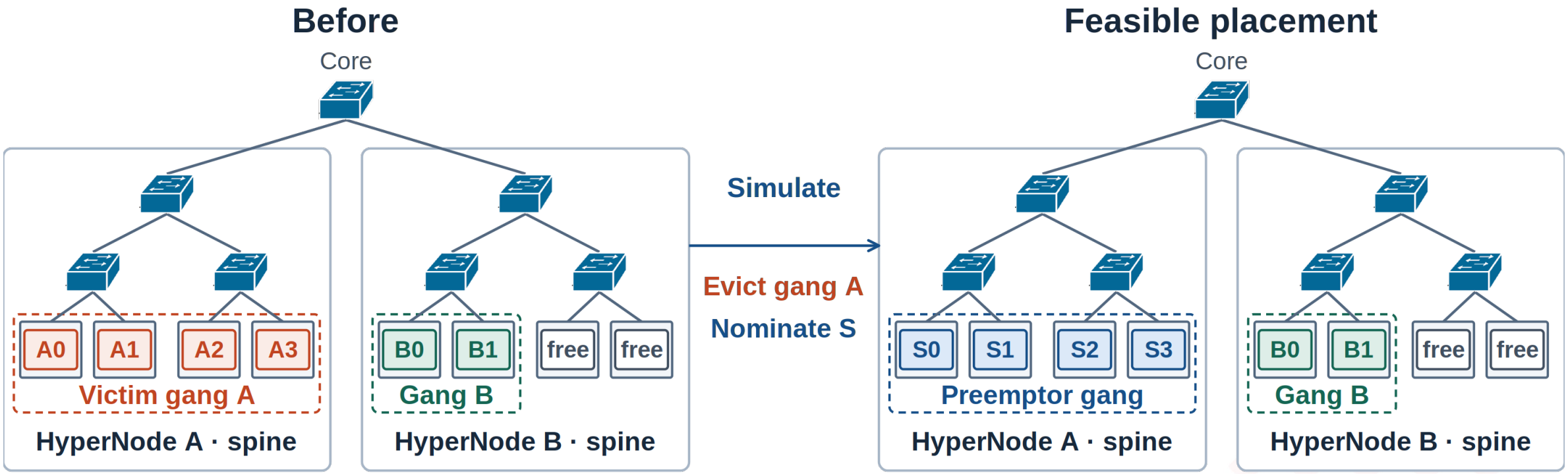
```
minMember: 8
networkTopology:
  mode: hard
  highestTierName: core
subGroupPolicy:
- name: prefill
  subGroupSize: 4
  networkTopology:
    highestTierName: leaf
- name: decode
  subGroupSize: 4
  networkTopology:
    highestTierName: spine
```

Gang Preemption Within a HyperNode

Preemptor gang
`minMember: 4`

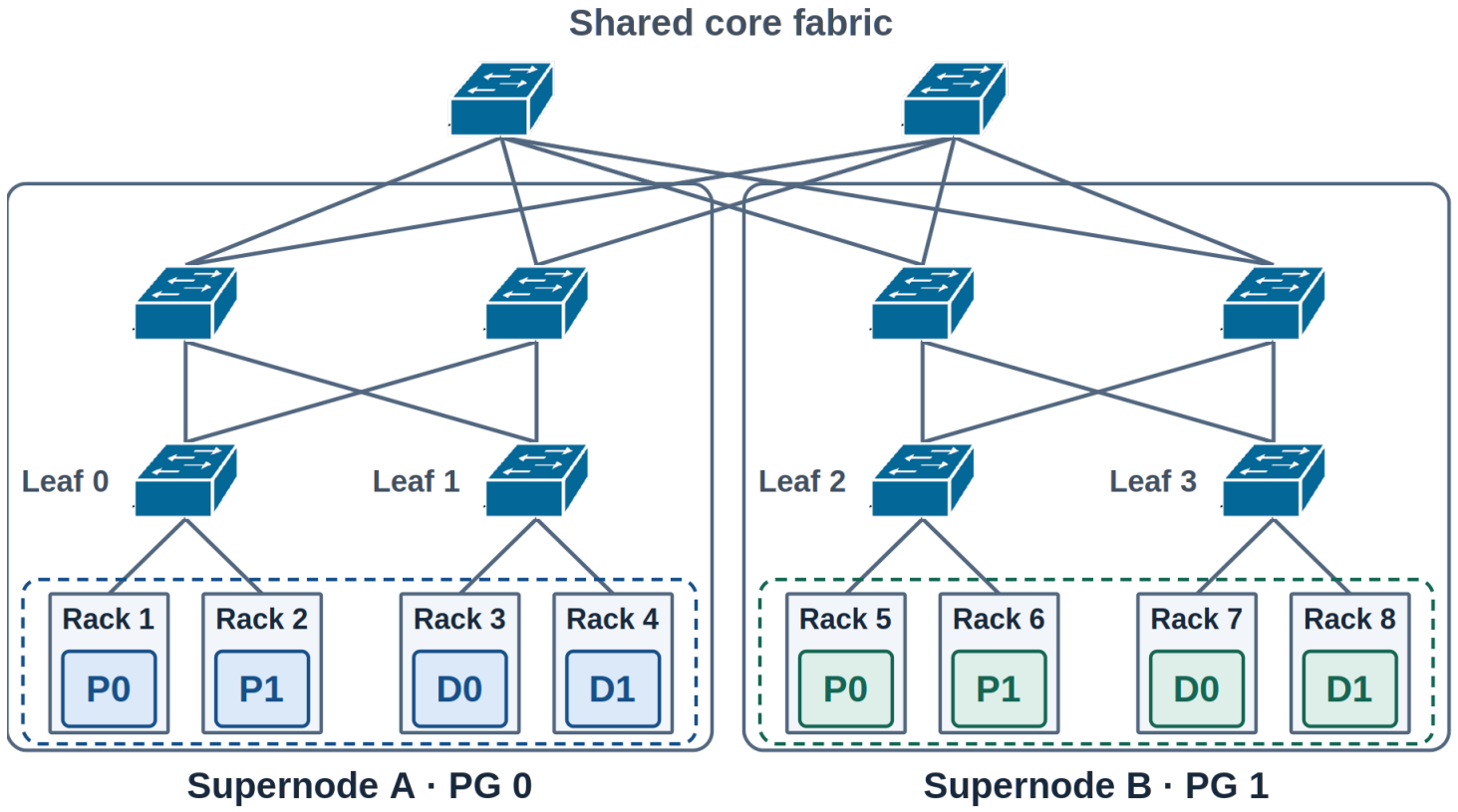
S0 S1 S2 S3

4 Pods within one spine HyperNode
`highestTierName: spine`



Keep Each Instance Together, Place Instances Apart

Same-label PodGroups → separate supernodes

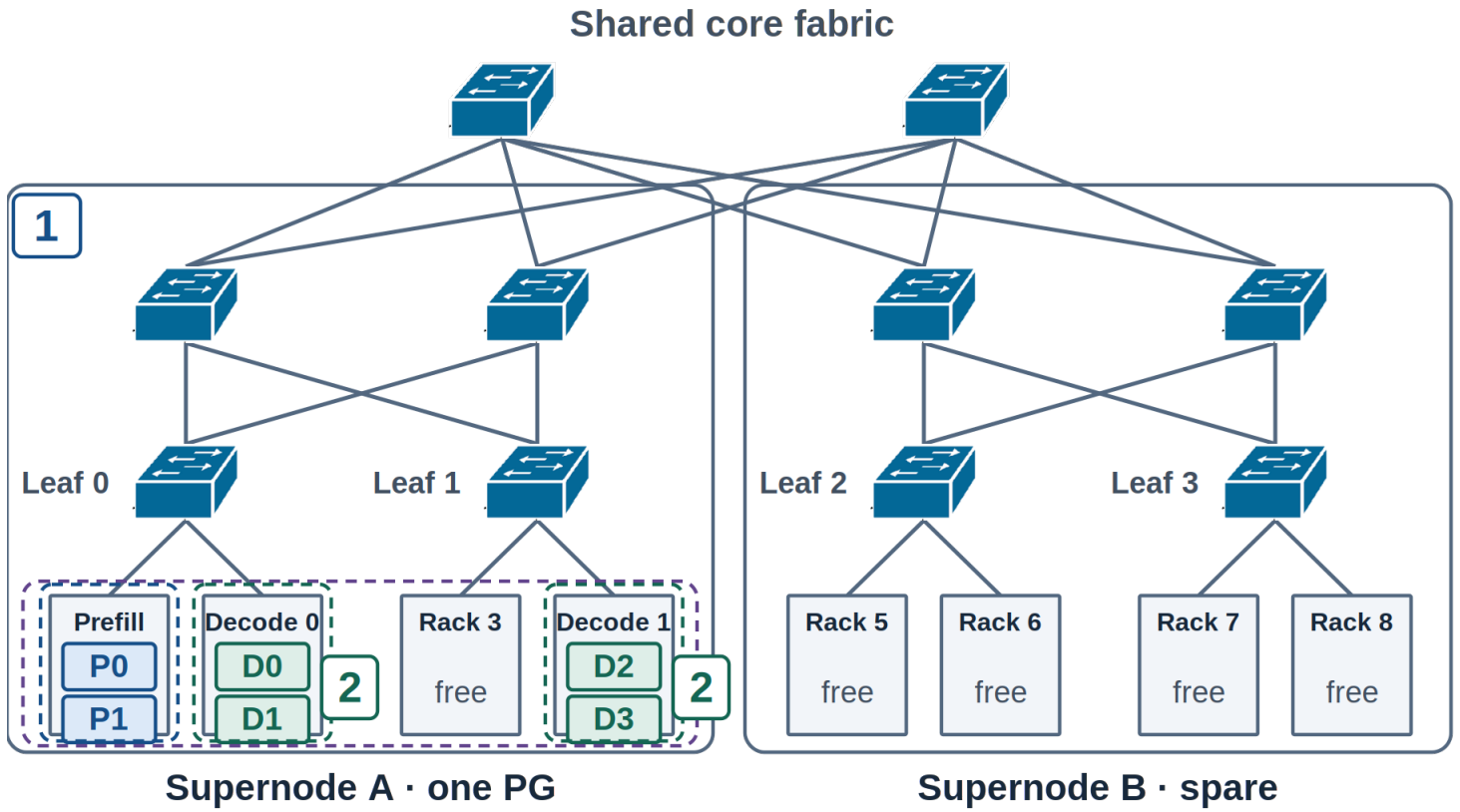


PodGroup.spec · key fields

```
networkTopology:
  mode: hard
  highestTierName: supernode
topologyAffinity:
  podGroupAntiAffinity:
    required:
    - podGroupSelector:
        matchLabels:
          app: model-a
      topologyTierName: supernode
```

PG label: app=model-a

Next: Relationships Between SubGroups



1 Affinity · same supernode

Prefill + Decode

2 Anti-affinity · separate leaves

Decode replica 0 / replica 1

03

Ecosystem

Workload controllers carry scheduling intent.

01 Unified scheduling

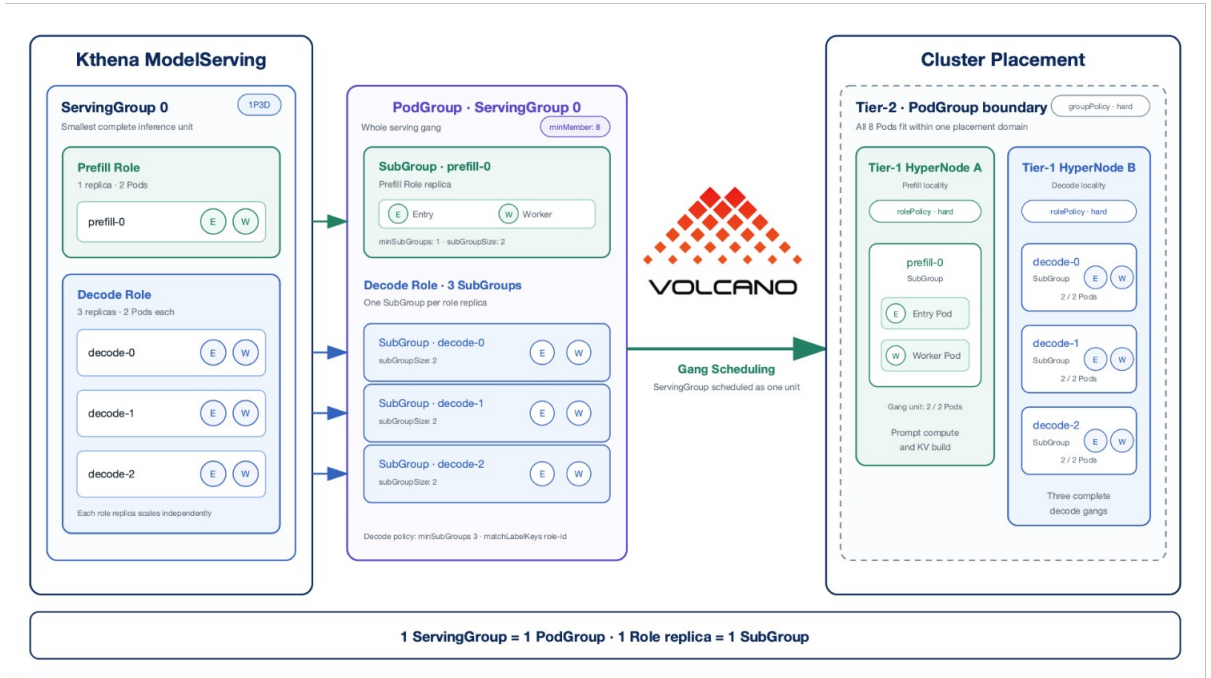
02 Multi-level gang & network topology

03 Ecosystem

04 Benchmark framework & progress

Kthena: Configure Group and Role Topology

ServingGroup / role replica



Group: spine / Role replica: leaf

ModelServing.spec · excerpts

```
schedulerName: volcano
replicas: 1
template:
  gangPolicy:
    minRoleReplicas:
      prefill: 1
      decode: 3
  networkTopology:
    groupPolicy:
      mode: hard
      highestTierName: spine
    rolePolicy:
      mode: hard
      highestTierName: leaf
```

Kthena: Carry Role Replicas into PodGroup

One ServingGroup → one PodGroup

ModelServing.spec.template

```
roles:
- name: prefill
  replicas: 1
  workerReplicas: 1
- name: decode
  replicas: 3
  workerReplicas: 1
```

1 entry + 1 worker → 2-Pod SubGroup

Role replica

SubGroup

PodGroup.spec · topology / gang

```
minMember: 8
networkTopology:
  mode: hard
  highestTierName: spine
subGroupPolicy:
- name: decode
  minSubGroups: 3
  subGroupSize: 2
  networkTopology:
    mode: hard
    highestTierName: leaf
```

KubeRay: Map Multi-Host Worker Replicas

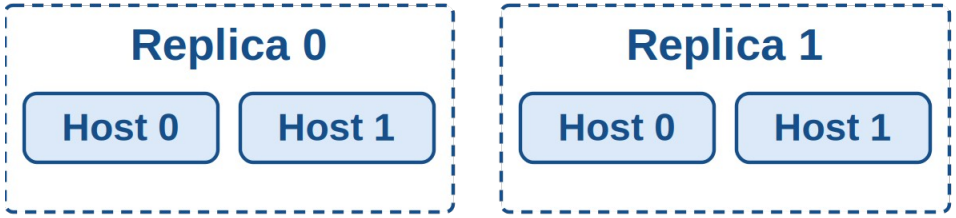
RAY Worker replica → complete SubGroup

RayCluster.spec · excerpt

```
enableInTreeAutoscaling: true
workerGroupSpecs:
- groupName: worker
  minReplicas: 2
  replicas: 2
  maxReplicas: 4
  numOfHosts: 2
```

2 replicas

2 hosts each



PodGroup.spec · worker excerpt

```
subGroupPolicy:
- name: worker
  minSubGroups: 2
  subGroupSize: 2
  labelSelector:
    matchLabels:
      ray.io/group: worker
  matchLabelKeys:
  - ray.io/worker-group-replica-name
```

Replica label keeps the hosts together

Grove: Map PodGang Groups into Volcano



PodGang → PodGroup

Grove PodGang.spec · excerpt

```
podGroups:
- name: prefill
  minReplicas: 2
- name: decode
  minReplicas: 4
```

2 Prefill + 4 Decode → minMember 6

Group → policy

Minimum Pods

Volcano PodGroup.spec · excerpt

```
minMember: 6
subGroupPolicy:
- name: decode
  subGroupSize: 4
  minSubGroups: 1
  labelSelector:
    matchLabels:
      grove.io/podclique: decode
  matchLabelKeys:
  - grove.io/podclique
```

Next: RBG, Kubeflow, and LWS



RBG

In progress

RoleInstance → SubGroup

Replica identity + gang constraints



Kubeflow

Planned

Training groups + topology

Trainer jobs → scheduling groups



LWS

Planned

Multi-role / multi-group

Leader / worker roles + group relations

04

Benchmark framework & progress

Measure behavior across workload shapes.

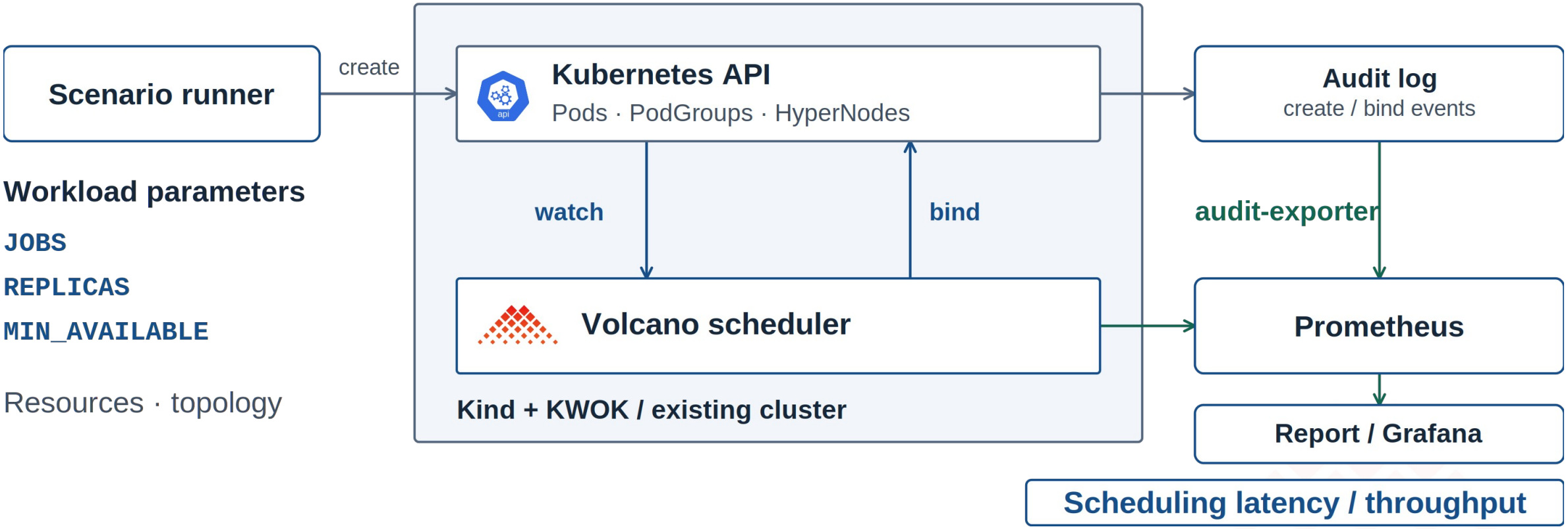
01 Unified scheduling

02 Multi-level gang & network topology

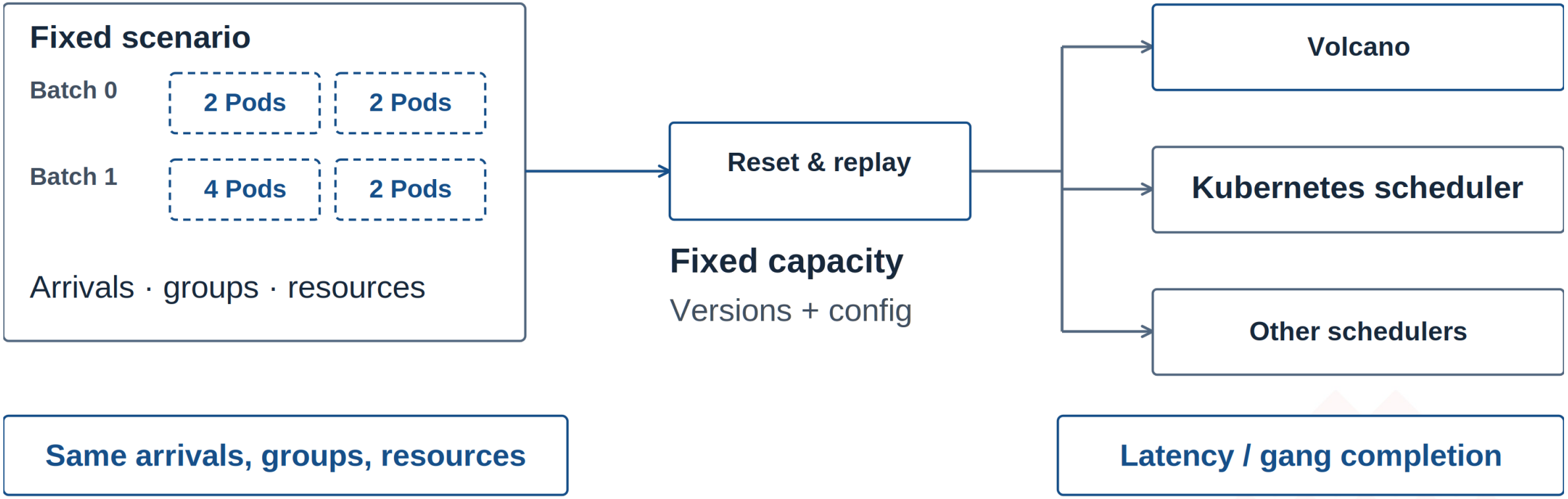
03 Ecosystem

04 Benchmark framework & progress

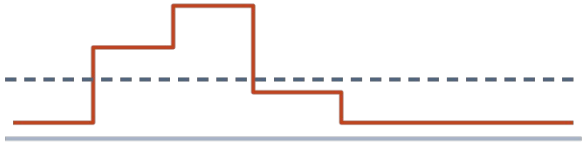
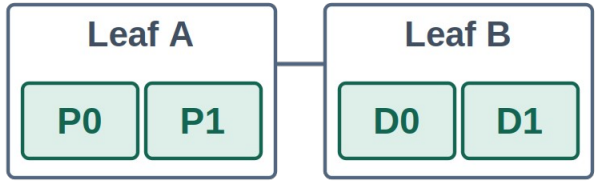
A Repeatable Scheduling Benchmark Framework



Next: Cross-Scheduler Comparisons



Next: Workload and Topology Test Matrix

Scenario		Variables	Metrics
Bursts		**Arrival rate · capacity** Pool capacity requests	**Tail latency** Backlog recovery
Gang size		**Group size · fragmentation** Large / small gangs	**Complete gangs** Binding throughput
Topology		**Tier limits · domains** Gang · anti-affinity	**Valid placement** Gang latency

v1.17: Placement, Preemption, and Performance

Preemption	**Fewer unnecessary evictions**	Gang admission + victim selection Topology-aware reclaim · in review
Group placement	**SubGroup affinity / anti-affinity**	Locality and replica isolation API and integration work · planned
Performance	**Cache locks + queue status cost**	Replay workloads and profile bottlenecks Cross-scheduler / topology tests · next

Volcano Community



Thank you

Questions & contributions

volcano.sh

GitHub

Slack